

Machine Learning — Lecture 2

Data Understanding, Preprocessing, and Feature Engineering

Dr. Fayçal Hamdi

Faculty of Computer and Information Systems

Lecture 2 at a Glance

Data Understanding, Preprocessing, and Feature Engineering

Course Learning Outcome

CLO2: Analyze datasets and apply appropriate preprocessing, transformation, and feature engineering techniques.

Lecture purpose

- Learn how to inspect data before modeling.
- Distinguish data types, feature types, and variable roles.
- Use summary statistics and visual tools to diagnose distributions.
- Select appropriate preprocessing and feature-engineering steps.
- Prevent leakage by fitting preprocessing only on training data.

Conceptual structure

Learning phase	Core question
1. Understand the data	What does the dataset represent and what do the variables mean?
2. Explore the data	What patterns, distributions, and relationships do we observe?
3. Diagnose data quality	What problems may make learning unreliable?
4. Build the representation	How should numerical and categorical features be transformed or engineered?
5. Build a valid workflow	How do we avoid leakage and adapt preprocessing to the model?

💡 Main idea: machine learning performance often depends as much on the **quality and representation of the data** as on the choice of model.

Learning Objectives

By the end of Lecture 2, you will be able to:

1. Distinguish numerical, categorical, ordinal, binary, and textual/image features.
2. Describe a dataset using observations, features, target variables, and metadata.
3. Compute and interpret mean, median, variance, standard deviation, quartiles, IQR, and outliers.
4. Use boxplots, histograms, scatter plots, and cross-tabulations to inspect data.
5. Explain missing values, duplicates, invalid entries, and inconsistent categories.
6. Apply scaling, normalization, standardization, encoding, and transformations appropriately.
7. Explain feature construction, feature selection, and feature leakage.
8. Design a preprocessing workflow that separates training, validation, and test information.

Lecture Roadmap

From raw data to model-ready representation

Learning phase	Parts	Core question
1. Understand the data	I	What does the dataset represent, and what do the variables mean?
2. Explore the data	II	What patterns, distributions, spread, skew, and relationships do we observe?
3. Diagnose data quality	III	What problems could make learning unreliable, and how should we respond?
4. Build the representation	IV–VI	How do we transform numerical features, encode categories, and engineer useful features?
5. Build a valid workflow	VII–VIII	How do we avoid leakage and choose preprocessing that fits the learning algorithm?

💡 **Part IX — Practice & consolidation:** diagnose feature types, detect leakage, recognize common mistakes, and justify preprocessing decisions.

Part I

Data Understanding

Before choosing a model, we need to understand what the dataset **represents**, what each variable means, and what problems the data may contain.

FCIS — For Course Use Only

Why Data Understanding Comes Before Modeling

A model learns from the representation it receives

A learning algorithm does not see the real-world problem directly.

It sees a table, matrix, image, text vector, or another representation.

Real Problem → Data Collection → Representation → Learning Algorithm

If the representation is poor, the model may learn the wrong pattern.

Examples

Data issue	Possible consequence
Important feature missing	Model cannot use a useful signal
Wrong unit or inconsistent scale	Distance/gradient-based methods may behave poorly
Categorical value misspelled	Same category split into different artificial groups
Target information leaked into features	Unrealistically high validation/test performance
Outliers untreated without justification	Model may be overly influenced by rare observations

💡 Good modeling begins with asking: **What exactly does each column mean, and can the model safely use it?**

Dataset Anatomy

Observations, variables, features, targets

A standard supervised-learning dataset is often represented as:

$$X \in \mathbb{R}^{n \times p}, \quad y \in \mathbb{R}^n$$

where:

- n is the number of observations;
- p is the number of features;
- each row is one example;
- each column is one measured attribute;
- y is the target variable.

Example — Student-performance prediction

Student	Study Hours	Attendance	Previous GPA	Learning Mode	Final Result
S1	12	92%	3.4	Group	Pass
S2	4	65%	2.1	Individual	Fail
S3	9	80%	2.8	Online	Pass

Here, **Study Hours**, **Attendance**, **Previous GPA**, and **Learning Mode** are features. **Final Result** is the target.

Feature Types

Not all variables behave the same way

Feature type	Example	Typical treatment
Continuous numerical	income, temperature, exam score	scaling, transformations, outlier analysis
Discrete numerical	number of orders, absences	may be scaled or treated as count data
Binary	has loan: yes/no	encode as 0/1
Nominal categorical	city, product type	one-hot or other categorical encoding
Ordinal categorical	low/medium/high	ordered encoding when order is meaningful
Date/time	timestamp, semester	extract useful components
Text	review, complaint	tokenize and vectorize
Image	handwritten digit	represent as pixel values or visual features

💡 The model does not automatically know the meaning of a variable. The representation must preserve the meaning needed for learning.

Variables Have Roles

A column can mean different things depending on the task

Consider a dataset of bank customers:

Customer	Age	Income	Existing Loan	Defaulted
C1	24	45000	Yes	No
C2	52	98000	No	No
C3	36	30000	Yes	Yes

If the task is default prediction

- Features: age, income, existing loan
- Target: defaulted

If the task is customer segmentation

- Features: age, income, existing loan
- No target is used
- The goal is to discover groups

The same table can support different machine-learning tasks depending on the objective.

Data Dictionary

The missing document that often determines success

A **data dictionary** explains each variable.

Field	What should be documented
Name	Column name used in the dataset
Meaning	What real-world quantity it represents
Type	numerical, categorical, date, text, image, etc.
Unit	SAR, minutes, cm, percentage, count
Range	valid minimum/maximum or category set
Missing rule	what a blank, zero, or special code means
Timing	when the value becomes available

Why timing matters

If a variable is only known **after** the event we want to predict, using it during training creates leakage.

💡 A feature must be available at prediction time. Otherwise, it may make the model look good in experiments but fail in real deployment.

First Questions to Ask About a Dataset

A practical checklist

1. What does each row represent?
2. What does each column mean?
3. What is the prediction or discovery task?
4. Which column, if any, is the target?
5. Which variables are numerical, categorical, ordinal, text, or image?
6. Are there missing, invalid, duplicate, or inconsistent values?
7. Are there outliers or extreme values?
8. Are the units and scales comparable?
9. Was the data collected before or after the target event?
10. Could any feature leak the answer?

A good data inspection can prevent hours of misleading modeling.

Part II

Exploratory Data Analysis

Exploratory Data Analysis helps us summarize, visualize, and question the data before fitting a model.

FCIS — For Course Use Only

Descriptive Statistics

Center and spread

Given values $x_1, x_2, \dots, x_n$

The sample mean is $\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$

The sample variance is $s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$

The sample standard deviation is $s = \sqrt{s^2}$

Interpretation

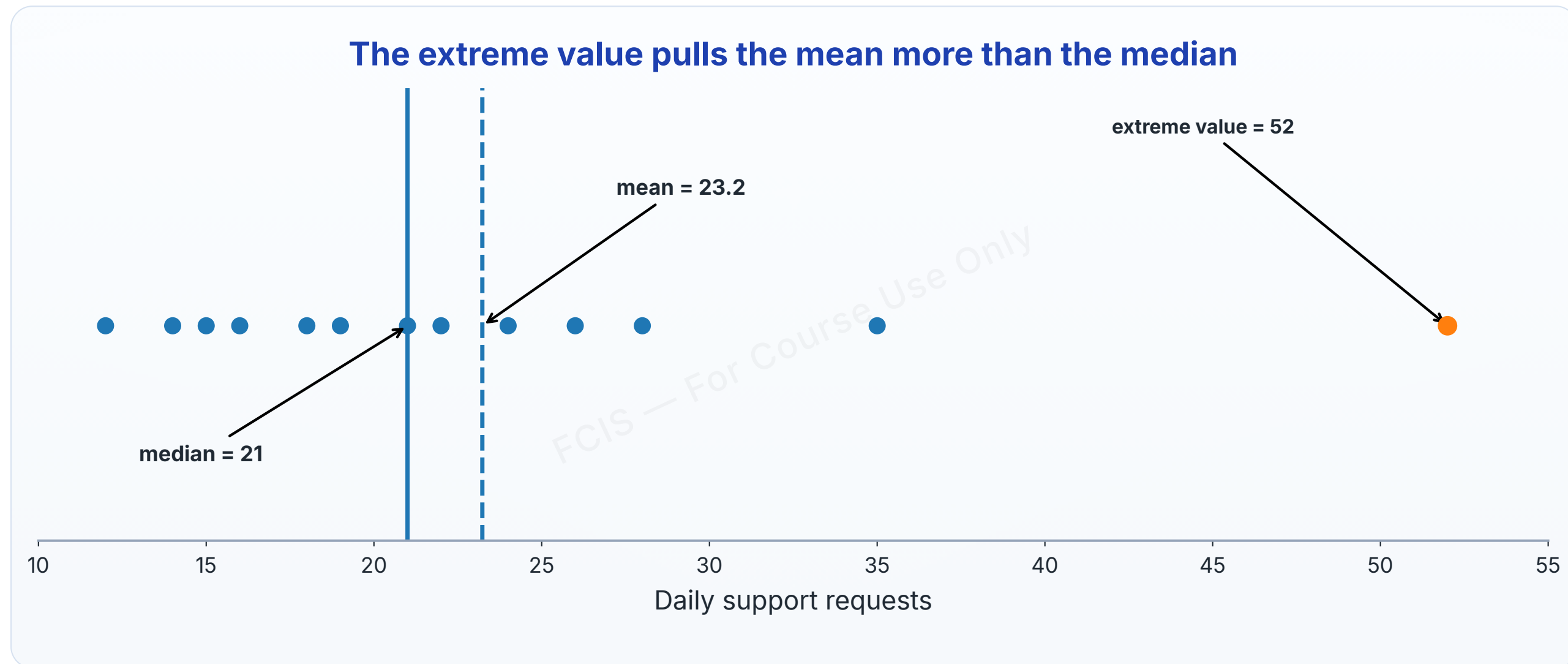
Statistic	Meaning
Mean	Balance point of the values
Median	Middle value after sorting
Standard deviation	Typical variation around the mean
Variance	Squared spread around the mean

💡 The mean is sensitive to extreme values. The median is often more robust when the distribution is skewed.

Mean vs. Median

Why robust summaries matter

The **mean** uses all numerical values and is sensitive to extreme observations. The **median** is the middle value after sorting and is more robust to outliers.



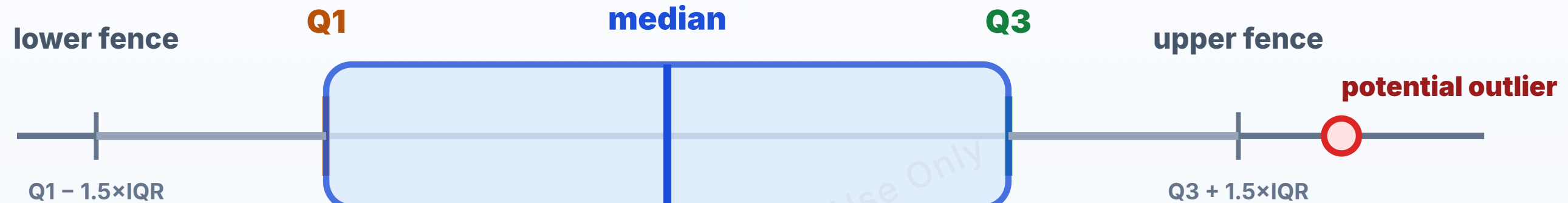
💡 **Takeaway.** When a distribution contains extreme values, the **median** often gives a more stable description of the center than the **mean**.

Quartiles and Interquartile Range

A compact description of distribution spread

Quartiles divide ordered data into four parts

Outside the fences, observations are often flagged for further inspection.



interquartile range (IQR) = Q3 - Q1

The IQR summarizes the spread of the middle 50% of the data.

The **interquartile range** is robust to extreme values because it focuses on the middle half of the distribution.

Worked Example: Quartiles and Outliers

Daily support requests

Sorted data: 12, 14, 15, 16, 18, 19, 21, 22, 24, 26, 28, 35, 52

Quartiles

There are 13 observations, so the median is the 7th value:

$$Q_2 = 21.$$

Lower half: 12, 14, 15, 16, 18, 19

Therefore:

$$Q_1 = \frac{15 + 16}{2} = 15.5.$$

Upper half: 22, 24, 26, 28, 35, 52

Therefore:

$$Q_3 = \frac{26 + 28}{2} = 27.$$

IQR and outlier fences

$$IQR = Q_3 - Q_1 = 27 - 15.5 = 11.5.$$

Lower fence:

$$Q_1 - 1.5(IQR) = 15.5 - 17.25 = -1.75.$$

Upper fence:

$$Q_3 + 1.5(IQR) = 27 + 17.25 = 44.25.$$

So:

- (12 > -1.75): no lower outlier.
- (35 < 44.25): 35 is still within the upper fence.
- (52 > 44.25): 52 is a potential outlier.

Result: (52) is flagged as a potential outlier. The value (35) is the largest observation still inside the fences.

💡 **Important:** the fences (-1.75) and (44.25) are **cutoff thresholds**, not necessarily whisker locations. In the corresponding boxplot, the whiskers end at the most extreme observed values within the fences: (12) and (35).

Boxplots

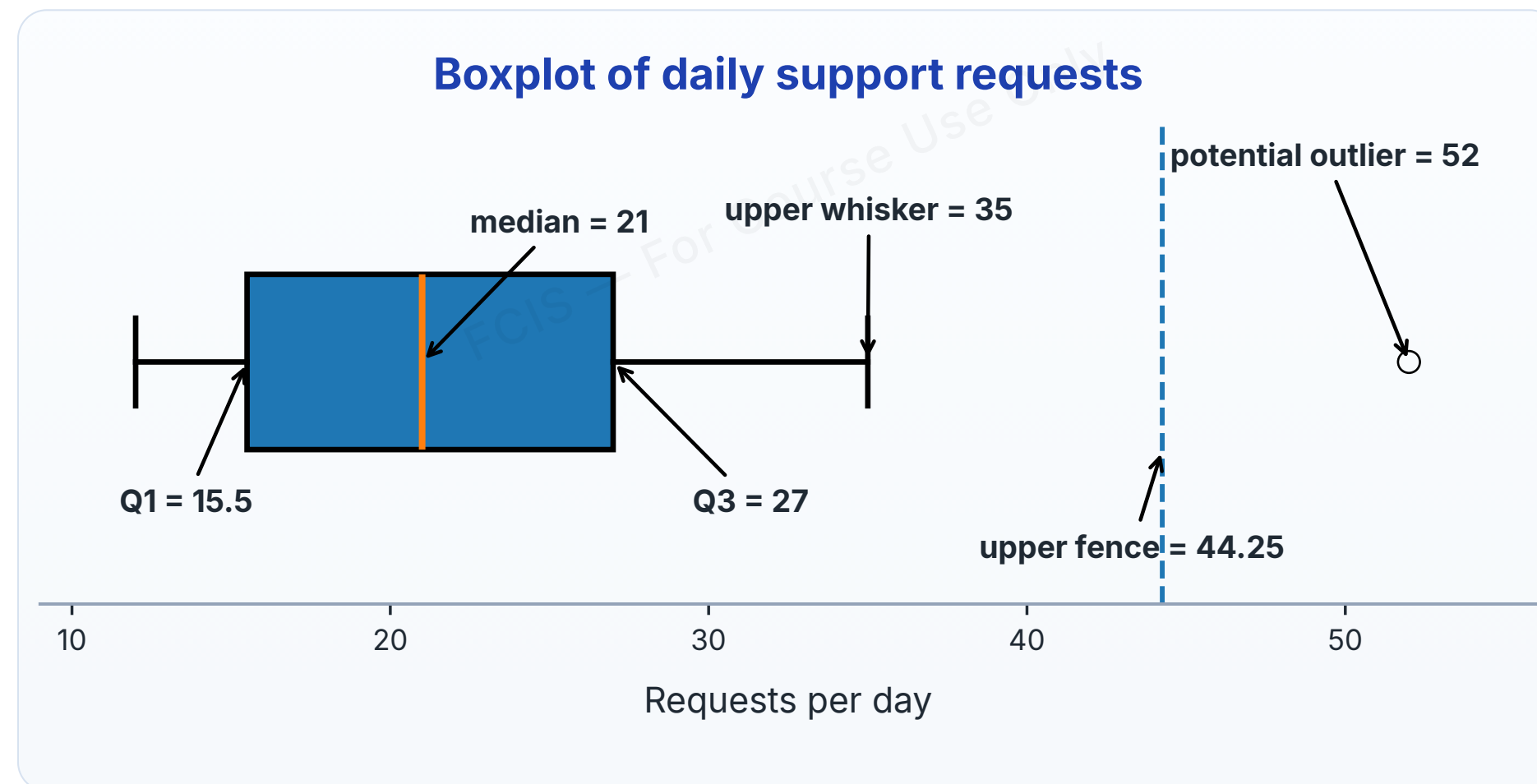
A visual summary of distribution

A **boxplot** summarizes a distribution with:

- the **box** from Q_1 to Q_3 ,
- a line for the **median**,
- **whiskers** extending to non-outlying observations,
- and separate marks for **potential outliers**.

In this example, the high value **52** appears beyond the upper whisker.

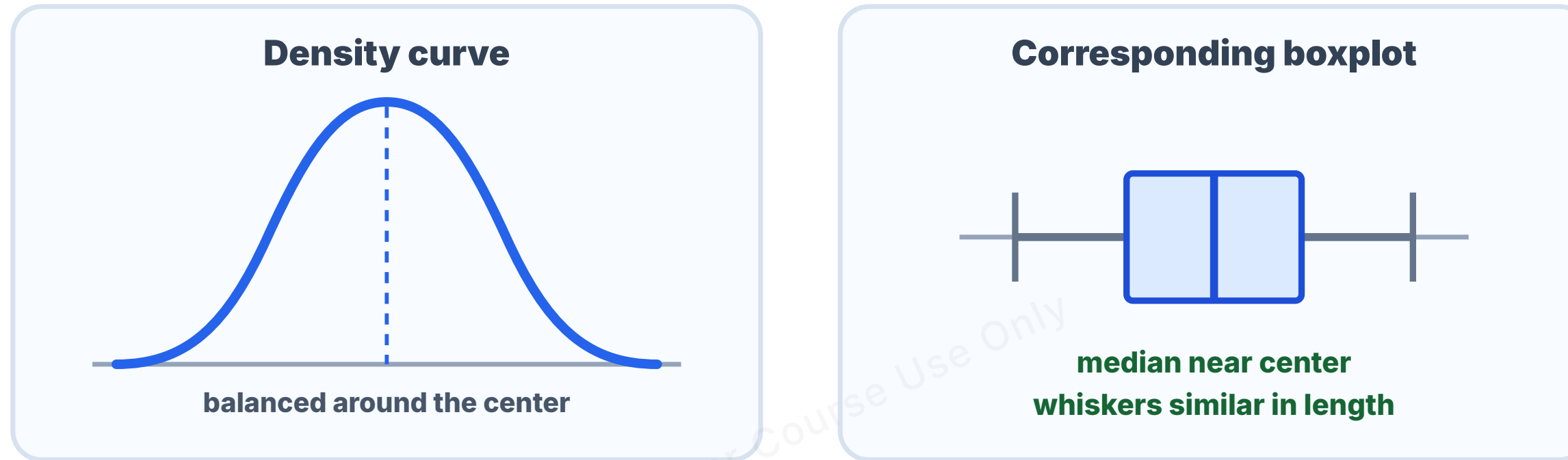
💡 Use boxplots to quickly compare center, spread, skew, and unusual observations.



Interpreting Boxplots

Match the boxplot to the distribution shape

Step 1 — symmetric distribution

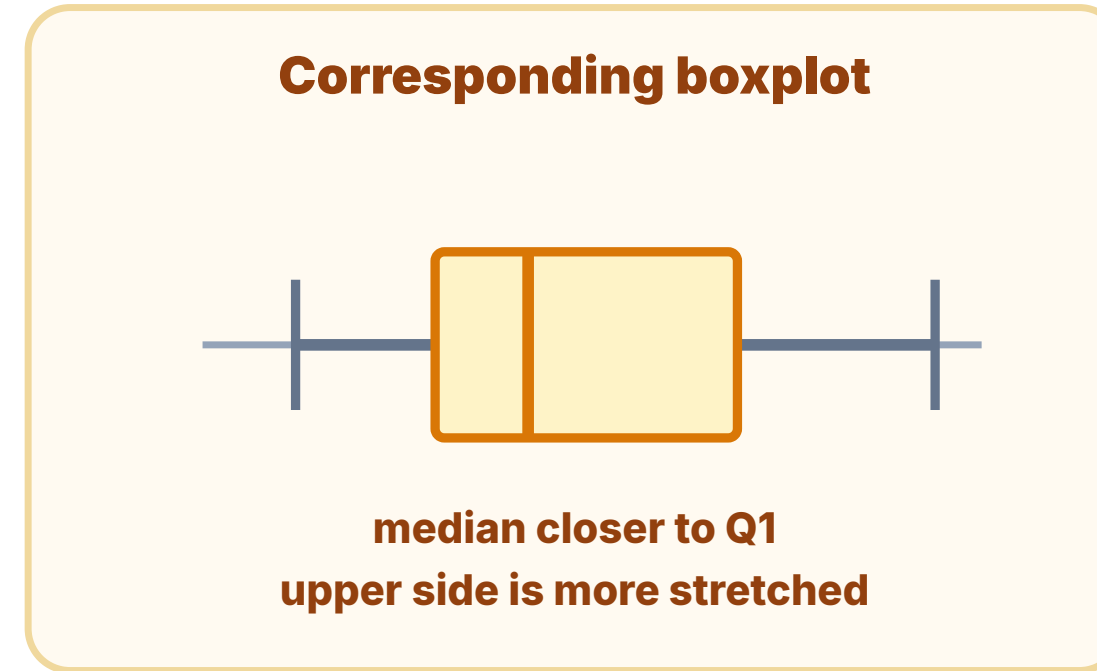
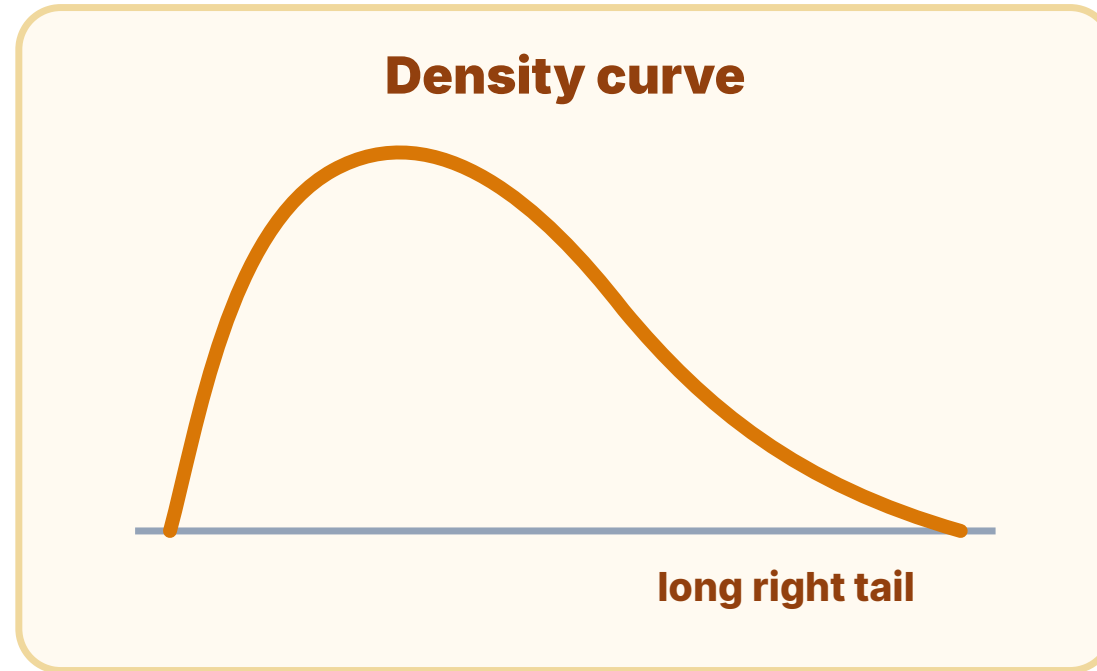


A symmetric boxplot usually has a centered median and similar spread on both sides.

A boxplot does not display the full density, but it can reveal **center**, **spread**, **asymmetry**, and potential **outliers**.

Match the boxplot to the distribution shape

Step 2 — right-skewed distribution

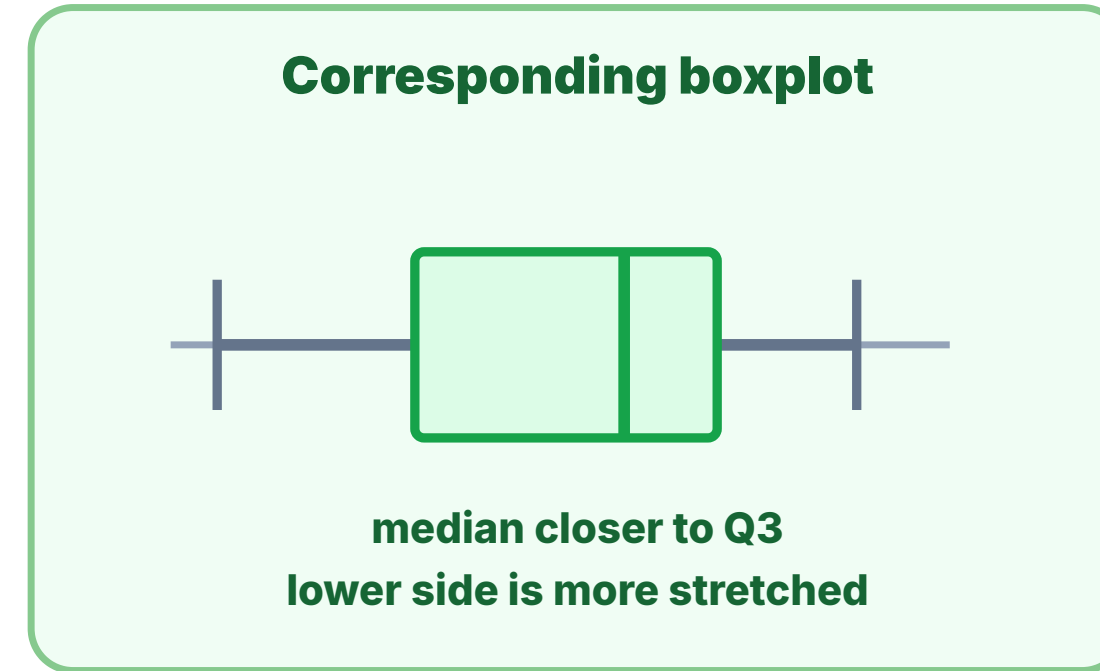
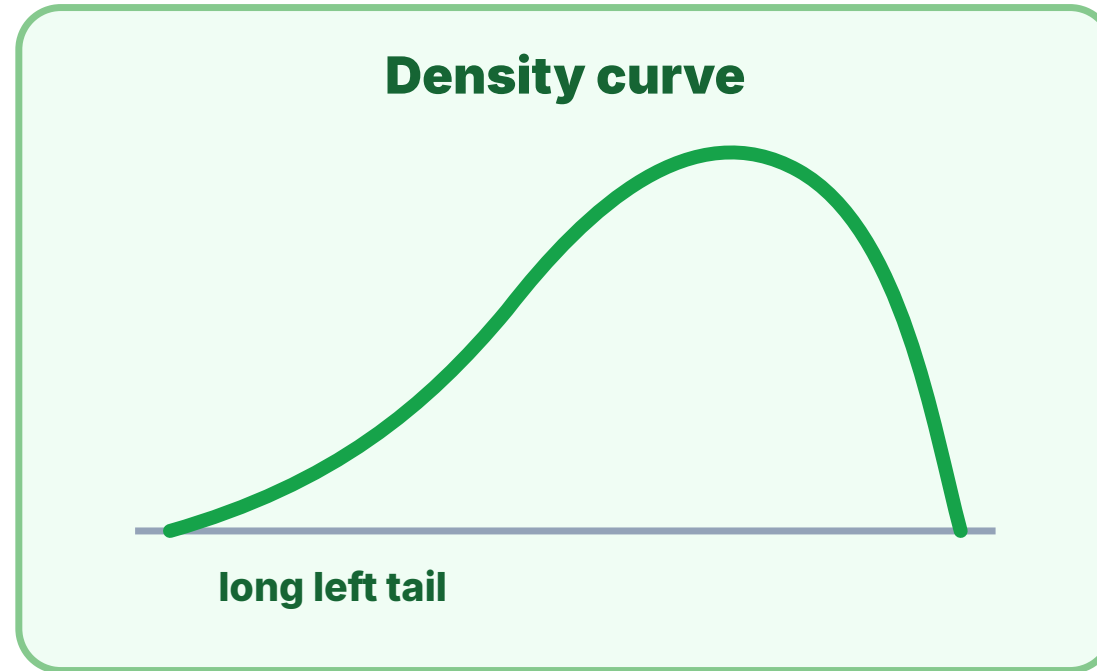


Right skew appears as more spread toward larger values; the upper whisker is often longer.

A boxplot does not display the full density, but it can reveal **center**, **spread**, **asymmetry**, and potential **outliers**.

Match the boxplot to the distribution shape

Step 3 — left-skewed distribution

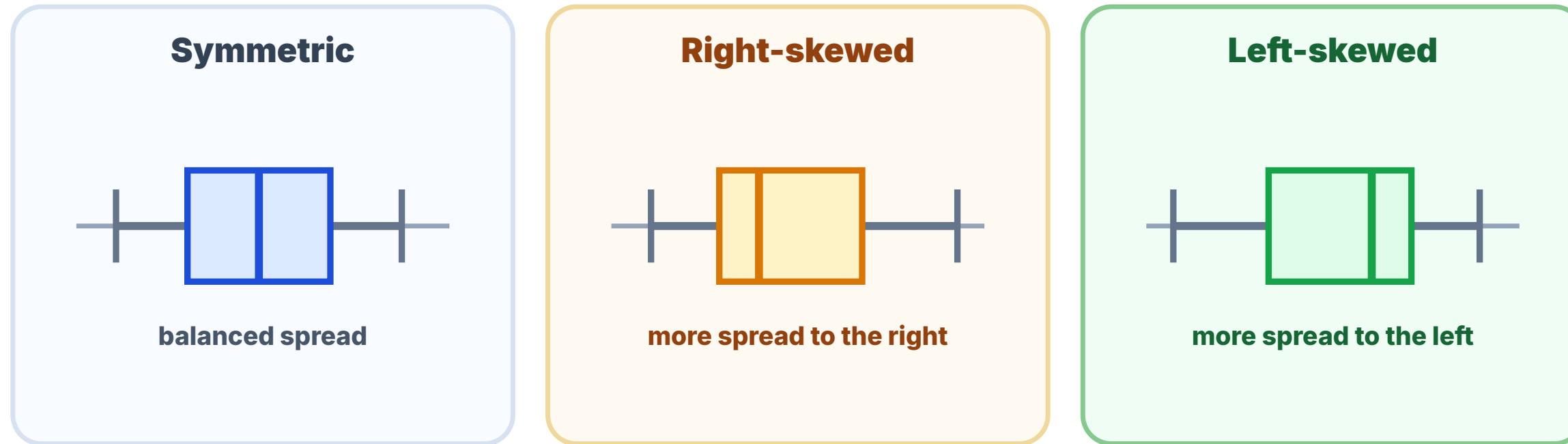


Left skew appears as more spread toward smaller values; the lower whisker is often longer.

A boxplot does not display the full density, but it can reveal **center**, **spread**, **asymmetry**, and potential **outliers**.

Match the boxplot to the distribution shape

Step 4 — compare the three patterns



Interpret the median position and relative spread together; no single visual cue is sufficient by itself.

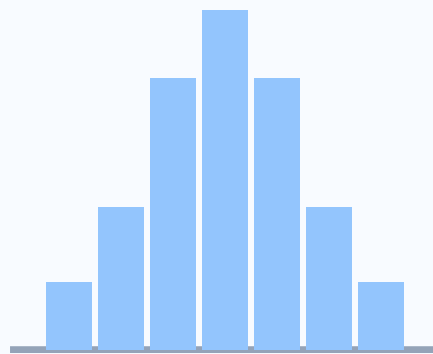
A boxplot does not display the full density, but it can reveal **center**, **spread**, **asymmetry**, and potential **outliers**.

Histograms and Distribution Shape

Seeing frequency patterns

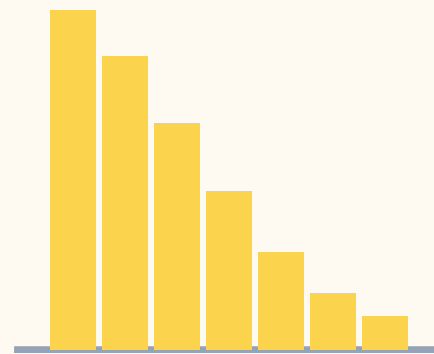
Common distribution shapes

Symmetric



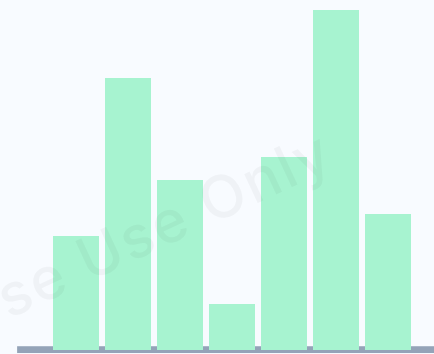
balanced around the center

Right-skewed



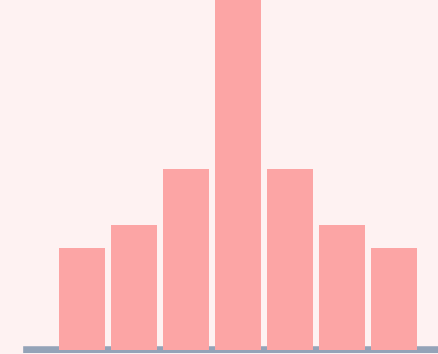
many small values, long right tail

Multimodal



more than one concentration region

Heavy-tailed



extreme values occur more often

💡 Histograms help us judge **shape**, **skew**, **spread**, and possible **subpopulations** in the data.

Income, spending, waiting time, and request counts are often right-skewed. A log transformation may sometimes make such features easier to model.

Scatter Plots

Inspecting relationships between two variables

A scatter plot helps detect:

- linear relationships;
- nonlinear relationships;
- clusters;
- outliers;
- heteroscedasticity;
- unusual groups.

Example questions

Question	What the plot may show
Does study time relate to exam score?	positive association
Does price increase linearly with area?	linear or nonlinear trend
Are there two separate populations?	clusters or subgroups
Does variance grow with the feature?	changing noise level

 Correlation or visual association does not automatically imply causation.

Correlation

Measuring linear association

The Pearson correlation coefficient measures the strength of a linear relationship between two numerical variables:

$$r = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_i (x_i - \bar{x})^2} \sqrt{\sum_i (y_i - \bar{y})^2}}$$

Interpretation:

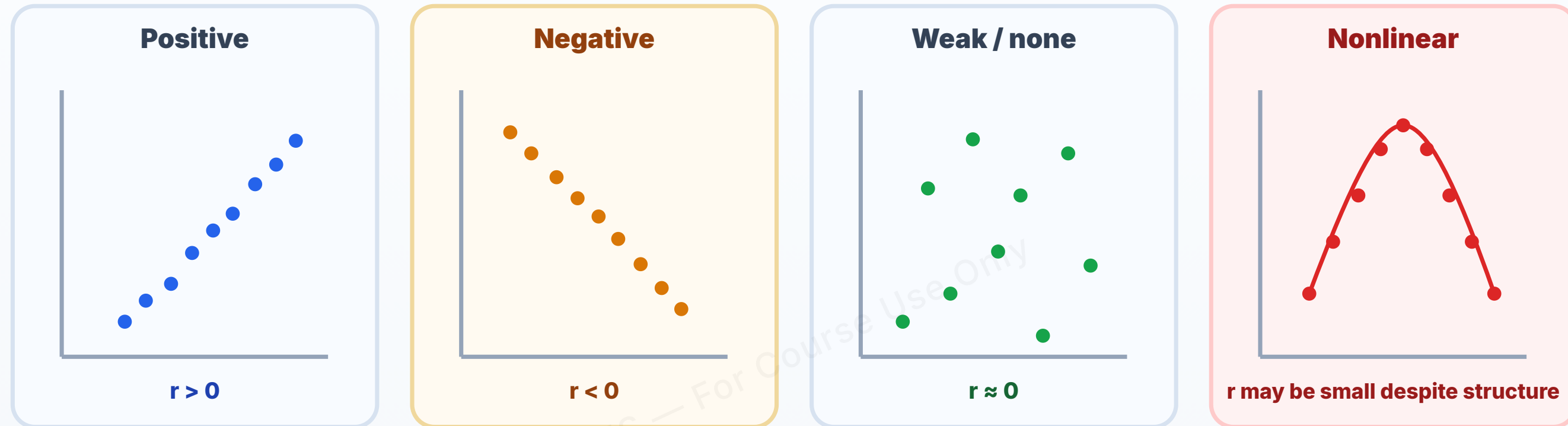
Value of r	Meaning
Close to $+1$	strong positive linear association
Close to 0	weak or no linear association
Close to -1	strong negative linear association

Correlation can miss nonlinear relationships. A scatter plot should often accompany correlation values.

Interpreting Correlation Patterns

Similar correlation values can hide very different relationships

Correlation summarizes linear association only



A low correlation does **not** always mean “no relationship.” It may indicate that the relationship is **nonlinear**.

Categorical Association

Cross-tabulations and independence

For categorical variables, a **contingency table** counts combinations of categories.

Example:

Study Method	Low Performance	High Performance	Total
Individual	18	42	60
Group	10	50	60
Online	32	28	60
Total	60	120	180

If two variables are independent, the expected count is: $E_{ij} = \frac{(\text{row total})(\text{column total})}{\text{grand total}}$

The Chi-square statistic is: $\chi^2 = \sum_{i,j} \frac{(O_{ij} - E_{ij})^2}{E_{ij}}$

💡 This test helps decide whether the observed differences are stronger than what independence would suggest.

Worked Example: Expected Counts

Study method and performance

For the first row and first column: $E_{\text{Individual, Low}} = \frac{60 \times 60}{180} = 20$

For the first row and second column: $E_{\text{Individual, High}} = \frac{60 \times 120}{180} = 40$

Since all row totals are 60, the expected counts are:

Study Method	Expected Low	Expected High
Individual	20	40
Group	20	40
Online	20	40

Contribution example

For Online-Low: $\frac{(32-20)^2}{20} = \frac{144}{20} = 7.2$

Large differences between observed and expected counts contribute strongly to χ^2 .

Worked Example: Chi-Square Statistic

Summing the contributions

Each cell contributes $\frac{(O_{ij} - E_{ij})^2}{E_{ij}}$:

Cell	O	E	Contribution
Individual-Low	18	20	0.2
Individual-High	42	40	0.1
Group-Low	10	20	5.0
Group-High	50	40	2.5
Online-Low	32	20	7.2
Online-High	28	40	3.6
Total			18.6

Chi-Square

$$\chi^2 = 0.2 + 0.1 + 5.0 + 2.5 + 7.2 + 3.6 = 18.6$$

Degrees of freedom

$$df = (r - 1)(c - 1) = (3 - 1)(2 - 1) = 2$$

Decision

Critical value at $\alpha = 0.05$: $\chi^2_{0.05, 2} = 5.99$

Since $18.6 > 5.99$, we **reject independence**.

💡 Study method and performance are **not independent** ($\chi^2 = 18.6$, $df = 2$, $p < 0.001$). The association is driven mainly by the **Low-performance** column — especially **Online** (7.2) and **Group** (5.0).

Part III

Data Quality Problems

Real datasets are imperfect. Preprocessing begins by distinguishing **data errors** from **meaningful but difficult observations**.

FCIS — For Course Use Only

Missing Values

Why a value may be absent

A missing value can mean different things:

Situation	Example	Possible treatment
Not recorded	age field left blank	impute or remove
Not applicable	spouse income for unmarried person	create explicit category/indicator
Sensor failure	temperature reading lost	impute using time/context
Hidden reason	high-income users skip income question	missingness may be informative

Common strategies

- remove rows when missingness is rare and random;
- remove features with excessive missingness;
- impute mean, median, mode, or learned values;
- add a missingness indicator;
- use models that can handle missing values.

💡 The correct treatment depends on why the value is missing, not only on how many values are missing.

Duplicates and Invalid Records

Not every row should be trusted

Duplicates

A duplicate may be:

- an accidental repeated entry;
- a legitimate repeated transaction;
- a repeated measurement;
- the same student/customer recorded under two IDs.

Invalid records

Examples:

Feature	Invalid value
Age	-3
Attendance	140%
Exam score	125 out of 100
Date	future date for past event
Category	M, Male, male, MALE as separate entries

Cleaning should be documented. Removing data without justification can be as harmful as keeping bad data.

Outliers

Error, rare event, or important signal?

An outlier is an observation that is unusually far from the rest of the data.

Possible interpretations

Interpretation	Example
Data entry error	age entered as 220
Measurement error	sensor spike
Rare but real event	unusually high hospital demand
Different population	corporate customer mixed with individual customers
Target-relevant signal	fraudulent transaction unusually large

Possible responses

- verify and correct if it is an error;
- keep if it is real and important;
- cap/winsorize extreme values;
- transform skewed features;
- use robust models or robust statistics;
- analyze separately if it represents a different group.

💡 Outlier treatment is a modeling decision and must be justified.

Inconsistent Categories

Small text differences can create artificial groups

Consider the following values for a feature `City`:

```
Madinah  
madinah  
Al Madinah  
Medina  
MADINAH
```

A model may treat these as different categories even when they refer to the same place.

Standardization steps

- trim spaces;
- standardize capitalization;
- map synonyms to one category;
- correct spelling variants;
- merge rare categories when justified.

Category cleaning is not cosmetic. It changes the feature representation that the model learns from.

Data Leakage

When the answer accidentally enters the features

Data leakage occurs when training or evaluation uses information that would not be available at prediction time.

Examples

Leakage example	Why it is invalid
Using <code>final_grade</code> to predict course pass/fail	it already contains the answer
Scaling using the full dataset before splitting	test information influences preprocessing
Choosing features after looking at test performance	test set becomes part of model selection
Using future transaction status to predict fraud	information arrives after the prediction moment

💡 Leakage can produce excellent experimental results that disappear in real deployment.

Part IV

Preprocessing Numerical Features

Preprocessing transforms raw variables into a form that learning algorithms can use reliably.

FCIS — For Course Use Only

Feature Scaling

Why scale matters

Some algorithms are sensitive to feature scale, especially those based on:

- distance;
- gradients;
- regularization;
- nearest neighbors;
- clustering;
- neural networks.

Example

Feature	Typical range
Annual income	20,000 to 300,000
Age	18 to 80
Number of dependents	0 to 8

Without scaling, income may dominate distance calculations simply because its numeric range is larger.

For KNN and K-Means, unscaled features can completely change nearest neighbors and clusters.

Standardization

Center and scale by standard deviation

Standardization transforms a feature into a z -score:

$$z = \frac{x - \mu}{\sigma}.$$

After standardization:

- the feature has mean approximately 0;
- the feature has standard deviation approximately 1.

When useful

- linear models with regularization;
- logistic regression;
- KNN;
- K-Means;
- neural networks;
- features with different units.

💡 The mean μ and standard deviation σ should be estimated from the training set only, then applied to validation/test data.

Min-Max Normalization

Rescale to a fixed interval

Min-max normalization maps a feature into a range such as $[0, 1]$:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Advantages

- easy to interpret;
- useful when a bounded range is desired;
- common for neural-network inputs and image intensities.

Caution

- sensitive to extreme values;
- new validation/test values may fall outside $[0, 1]$ if they exceed the training range.

Standardization is often more robust than min-max scaling when the feature contains outliers.

Transforming Skewed Features

Making patterns easier for the model

Highly skewed numerical features can be difficult to model directly. A transformation such as:

$$x' = \log(1 + x)$$

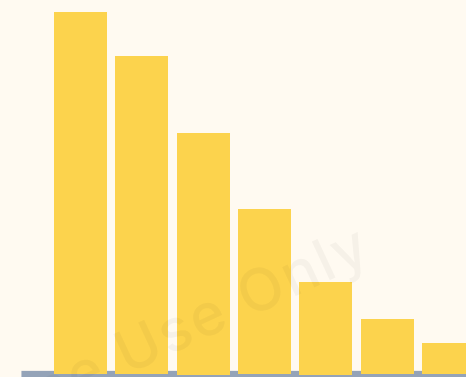
compresses very large values while preserving the ordering of observations.

Typical benefits:

- reduced skew,
- less influence of extreme large values,
- patterns that are easier for some models to use.

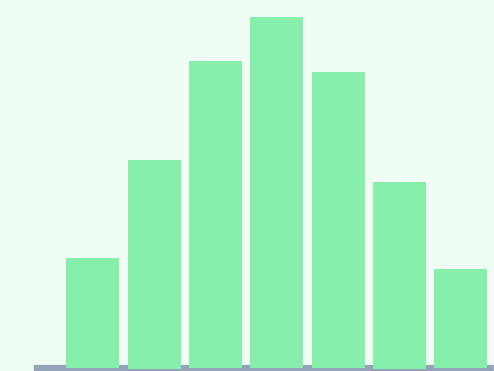
A log transform compresses the long right tail

Raw feature



strong right-skew

After $x' = \log(1 + x)$



spread becomes more balanced

💡 Transformations change the **representation**, not the underlying cases. They are useful when the original scale hides patterns or creates instability.

Binning and Discretization

Turning numerical values into intervals

Binning groups continuous values into categories.

Example:

Age	Age group
19	young
34	adult
67	senior

Possible benefits

- simplifies nonlinear relationships;
- improves interpretability;
- handles noisy measurements;
- aligns with domain rules.

Possible cost

- loses numerical detail;
- bin boundaries may be arbitrary;
- can create discontinuities.

Binning can be useful for reports and decision rules, but it should not be used without considering information loss.

Part V

Encoding Categorical Features

Most machine-learning algorithms require numerical inputs, so categorical variables must be encoded carefully.

FCIS — For Course Use Only

Label Encoding

Useful only when order is meaningful

Suppose:

Low, Medium, High

If the order is meaningful, we may encode:

$$Low \rightarrow 0, \quad Medium \rightarrow 1, \quad High \rightarrow 2.$$

This can be appropriate for ordinal variables.

Danger

For nominal categories:

Red, Blue, Green

encoding as:

$$Red \rightarrow 0, \quad Blue \rightarrow 1, \quad Green \rightarrow 2$$

creates an artificial order and distance.

💡 Do not use numerical codes for nominal categories if the model may interpret the numbers as ordered or metric.

One-Hot Encoding

A safer representation for nominal categories

For a feature **Learning Mode** with categories:

Individual, Group, Online

one-hot encoding creates binary features:

Learning Mode	Individual	Group	Online
Individual	1	0	0
Group	0	1	0
Online	0	0	1

Advantages

- no false numerical order;
- clear interpretation;
- works well with many standard models.

Caution

- increases dimensionality;
- rare categories may create sparse features;
- unseen categories require a defined handling strategy.

High-Cardinality Categories

When one-hot encoding becomes large

A categorical feature has high cardinality if it has many distinct values.

Examples:

- product ID;
- user ID;
- postal code;
- course code;
- device ID.

Possible strategies

- group rare categories;
- use domain-based grouping;
- encode frequencies carefully;
- use target encoding only with strict cross-validation;
- learn embeddings in more advanced models;
- remove the feature if it is not meaningful.

💡 High-cardinality features can easily memorize training examples, especially when categories are nearly unique.

Text, Images, and Structured Inputs

Different raw data require different representations

Raw data	Simple representation	Later course connection
Text review	bag-of-words counts	NLP features and language models
Image	pixel intensities	computer vision and neural networks
Time series	lagged values and rolling statistics	forecasting and sequential models
Transaction history	aggregated behavior features	customer modeling and fraud detection
Graph data	node/edge features	network analysis

Feature engineering is often the process of turning complex raw data into structured variables that a model can learn from.

Part VI

Feature Engineering

Feature engineering creates, transforms, or selects variables so that the model can represent the important pattern more easily.

FCIS — For Course Use Only

Feature Construction

Creating useful variables from existing ones

Examples:

Raw variables	Engineered feature
purchase dates	days since last purchase
total amount and number of orders	average order value
height and weight	body-mass index
timestamp	hour, day of week, semester
text message	number of exclamation marks
transaction sequence	number of failed attempts in last hour

💡 A good feature often encodes domain knowledge that the model would otherwise have difficulty discovering.

Interaction Features

When the effect of one variable depends on another

A model may need to represent interactions.

Example:

- `study hours` may matter differently depending on `attendance`;
- `loan amount` may matter differently depending on `income`;
- `age` may matter differently depending on `medical condition`.

One simple interaction feature is:

$$x_{\text{interaction}} = x_1 x_2$$

Why this helps

A linear model using x_1 , x_2 , and $x_1 x_2$ can represent some relationships that cannot be captured by using x_1 and x_2 separately.

Feature engineering can increase model flexibility without changing the learning algorithm.

Polynomial Features

A simple way to represent curvature

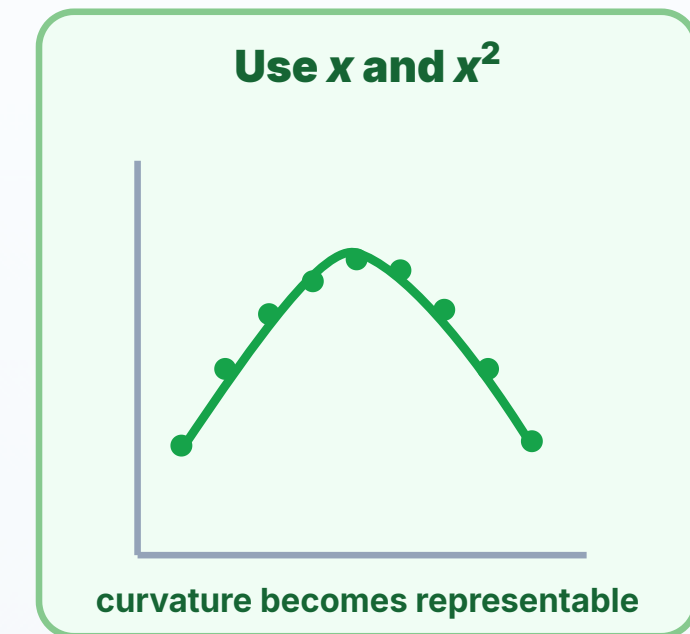
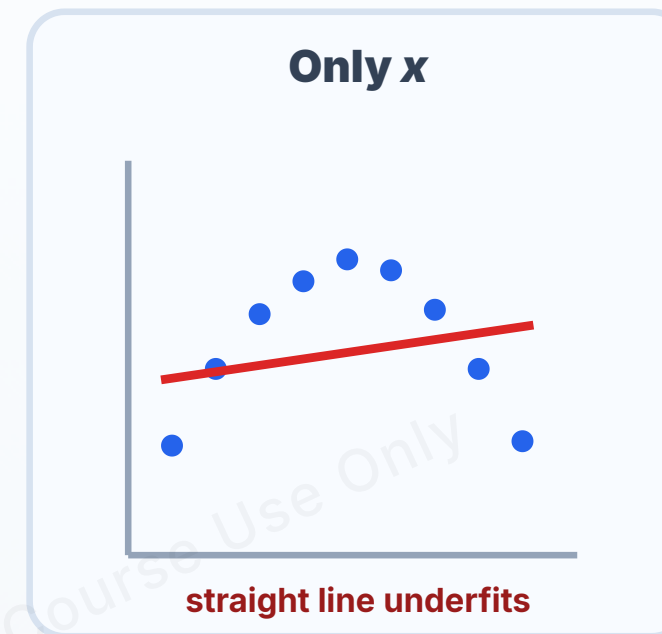
A linear model in the original feature x can only learn a straight-line relation.

By expanding the representation to include x^2 or x^3 , the model can express curved patterns while remaining linear in its parameters.

$$\hat{y} = \beta_0 + \beta_1 x + \beta_2 x^2$$

This is a change in **feature representation**.

Polynomial features let a simple model capture curvature



Adding polynomial features increases flexibility. This can help, but it may also increase the risk of **overfitting** if complexity grows too much.

Feature Selection

Removing what does not help

Feature selection means choosing a subset of available features.

Reasons to select features

- reduce overfitting;
- improve interpretability;
- reduce computation;
- remove redundant variables;
- avoid leakage;
- simplify deployment.

Common signs a feature may be problematic

- nearly constant;
- duplicate of another feature;
- available only after the prediction event;
- extremely high missingness;
- encodes an ID rather than a meaningful pattern;
- strongly correlated with the target for suspicious reasons.

A feature that gives very high accuracy may be excellent — or it may be leaked information.

Feature Engineering vs. Model Choice

Representation and algorithm work together

Some models need explicit features.

Others can learn more complex representations automatically.

Situation	Feature engineering role
Linear/logistic regression	often important for nonlinearities and interactions
KNN/K-Means	scaling and distance representation are critical
Decision trees	less sensitive to monotonic scaling, but still affected by irrelevant features
Neural networks	can learn representations, but still need clean and well-scaled inputs
Text/image models	representation choice is often the central modeling decision

💡 A better model cannot fully compensate for a poor representation or leaked evaluation process.

Part VII

Building a Leakage-Safe Preprocessing Workflow

The most important rule: transformations that learn from data must be fitted using the training set only.

FCIS — For Course Use Only

The Correct Development Workflow

Fit transformations on training data only

Suppose we standardize a feature.

Correct workflow

1. Split data into training, validation, and test sets.
2. Estimate μ and σ using the training set.
3. Transform training data using those values.
4. Transform validation and test data using the same training-set values.
5. Select models using validation performance.
6. Use the test set only once for final evaluation.

$$\mu_{\text{train}}, \sigma_{\text{train}} \implies z = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}}$$

💡 Validation/test examples must simulate future unseen examples. They should not influence preprocessing decisions.

Incorrect Workflow: Leakage

A common but serious error

Leakage workflow

1. Compute preprocessing statistics on the full dataset.
2. Split into training and test sets.
3. Train the model.
4. Report test accuracy.

Why invalid?

The test distribution influenced the preprocessing parameters.

Even if labels were not used, information from the test examples entered the training process.

This is especially problematic for imputation, scaling, feature selection, and target encoding.

Preprocessing Pipeline

Keep transformations reproducible

A pipeline records the sequence of transformations.

Example:

Raw Data → Imputation → Scaling → Encoding → Model

Benefits

- reproducible experiments;
- fewer manual errors;
- consistent training and validation/test transformations;
- easier deployment;
- clearer separation of preprocessing and modeling steps.

💡 A pipeline is not only software convenience. It is a way to preserve correct experimental discipline.

Worked Workflow Example

Predicting apartment rental price

Dataset columns:

Column	Type	Possible preprocessing
Area	numerical	scaling, check outliers
Number of rooms	discrete numerical	scaling or keep as count
District	nominal categorical	one-hot encoding
Building age	numerical	scaling, possible nonlinear effect
Furnished	binary	encode 0/1
Monthly rent	target	no preprocessing as a feature

Feature construction ideas

- price per square meter for analysis;
- age group;
- room-density feature;
- district grouping if many rare categories.

The target **Monthly rent** must not be included among the input features when training the model.

Worked Workflow Example

Fraud detection features

Possible raw fields:

Field	Potential feature	Leakage risk?
Transaction amount	scaled amount or log amount	low
Merchant category	one-hot encoded category	low
Country	encoded country	low
Number of failed attempts before transaction	count feature	low if available before decision
Chargeback status	do not use	high leakage
Investigation result	do not use	high leakage
Time since previous transaction	engineered time feature	low

💡 For predictive modeling, the time at which a feature becomes known is as important as the feature itself.

Part VIII

Matching Preprocessing to the Model

Preprocessing choices should be linked to the assumptions and mechanics of the learning algorithm.

FCIS — For Course Use Only

Linear and Logistic Regression

Preprocessing priorities

Linear and logistic regression often benefit from:

- numerical scaling;
- meaningful encoding of categorical variables;
- handling multicollinearity;
- feature transformations for nonlinear relationships;
- interaction features when domain knowledge suggests them;
- regularization when many features are used.

Why scaling matters

If regularization is used, the penalty applies to coefficients.

Unscaled features can make coefficient magnitudes difficult to compare.

A coefficient for income measured in SAR and a coefficient for age measured in years are not directly comparable without considering scale.

KNN and K-Means

Distance-based methods are highly scale-sensitive

Both KNN and K-Means use distances.

If features are on very different scales, one feature may dominate:

$$d(x, z) = \sqrt{\sum_j (x_j - z_j)^2}$$

Example

Feature	Difference
Income	50,000
Age	5
Number of orders	3

Without scaling, income dominates the distance.

💡 Scaling is not optional for distance-based methods unless all features are already meaningfully comparable.

Decision Trees

Less scale-sensitive, but not preprocessing-free

Decision trees split based on feature thresholds or category partitions.

They are usually:

- less sensitive to monotonic scaling;
- able to handle nonlinear feature relationships;
- interpretable through decision rules.

But they still require attention to:

- missing values;
- categorical encoding depending on implementation;
- noisy features;
- leakage;
- rare categories;
- overfitting through deep trees.

A tree may not care whether age is measured in years or standardized units, but it can still overfit to noisy or leaked features.

Neural Networks

Clean numerical input helps training

Neural networks usually benefit from:

- scaled numerical features;
- consistent encoding;
- removal or correction of invalid values;
- careful treatment of missing data;
- enough data for the model complexity;
- train/validation monitoring.

FCIS — For Course Use Only

Why scaling helps

Gradient-based training can be unstable or slow when input features have very different scales.

💡 Good preprocessing makes optimization easier and often improves generalization.

Part IX

Practice, Diagnosis, and Common Mistakes

Preprocessing is not a mechanical checklist. Each decision should be justified by the data, the task, and the model.

FCIS — For Course Use Only

Activity: Diagnose Feature Types

Decide the appropriate treatment

A dataset for predicting whether a customer will default contains:

Feature	Example value	Type	Possible treatment
Age	37		
Employment status	Employed		
Credit score	710		
Existing loan	Yes		
City	Madinah		
Application date	2026-03-14		
Default	No		

Questions

1. Which column is the target?
2. Which features require encoding?
3. Which features may require scaling?
4. What useful features could be extracted from the date?

Possible reasoning: **Default** is the target; employment status, existing loan, and city need categorical encoding; age and credit score may be scaled; month, weekday, or time since previous application may be derived if meaningful.

Activity: Find the Leakage

Which feature should not be used?

A hospital wants to predict whether a patient will be readmitted within 30 days.

Available columns:

Column	Description
Age	patient age at admission
Diagnosis	initial diagnosis
Length of stay	number of days in hospital
Discharge medication count	number of medications at discharge
Readmission date	date of next admission if any
Previous admissions	number of admissions before current one
Follow-up scheduled	whether follow-up was scheduled before discharge

Questions

1. Which column is the most obvious leakage feature?
2. Which features depend on the prediction time?
3. What should be available when the model makes its prediction?

Readmission date directly reveals the target and should not be used. Whether **length of stay** is valid depends on when the prediction is made: at admission or at discharge.

Activity: Choose a Transformation

Match the problem to the preprocessing step

Situation	Candidate preprocessing
Numerical features have very different ranges	
Nominal variable has three categories	
Feature is strongly right-skewed	
Important variable is missing in 5% of records	
Target value appears as a hidden input feature	
A date field may contain useful seasonality	

Possible answers: scaling; one-hot encoding; log transformation; imputation with justification; remove leaked feature; extract month/day/semester features.

Common Mistakes

Avoid these preprocessing errors

Mistake	Why it is a problem
Fitting scaler before the train/test split	leaks information from validation/test data
Encoding categories inconsistently across splits	feature columns no longer match
Removing all outliers automatically	may remove rare but important cases
Treating nominal categories as ordered numbers	creates false distances and order
Ignoring missingness mechanism	may hide informative patterns
Including IDs as features without justification	may encourage memorization
Choosing features after seeing test results	turns the test set into a validation set
Reporting model performance without documenting preprocessing	makes the result difficult to reproduce

Mini-Quiz

Check your understanding

1. Why should we inspect a dataset before fitting a model?
2. What is the difference between a nominal and ordinal categorical variable?
3. Why is the median often preferred when a distribution has extreme values?
4. What does the IQR measure?
5. Why does KNN require careful feature scaling?
6. Give one example of data leakage.
7. Why should preprocessing parameters be estimated from the training set only?
8. What is one advantage and one disadvantage of one-hot encoding?

Suggested answers: to understand variables and data quality; nominal has no order while ordinal has meaningful order; median is robust to extremes; IQR is $Q_3 - Q_1$; KNN uses distance; using the target-derived field as input is leakage; validation/test must simulate unseen data; one-hot avoids false order but increases dimensionality.

Lecture 2 Summary

Key ideas to retain

Data understanding

- Identify observations, variables, features, targets, and data types.
- Use summaries and plots to understand center, spread, shape, and relationships.
- Mean and standard deviation are useful, but median and IQR are more robust to extremes.
- Boxplots and histograms help reveal skewness and outliers.
- Cross-tabulations and Chi-square reasoning help inspect categorical association.

Preprocessing and features

- Missing values, duplicates, invalid entries, and inconsistent categories require careful treatment.
- Scaling is essential for many distance-based and gradient-based models.
- Categorical variables require appropriate encoding.
- Feature engineering can make important patterns easier to learn.
- Preprocessing must be fitted using training data only to avoid leakage.

End-of-Lecture Check

Five questions to test your understanding

1. What information should a data dictionary contain, and why does timing matter?
2. How do mean, median, standard deviation, quartiles, and IQR describe a numerical feature differently?
3. Why should an outlier not be removed automatically?
4. When is one-hot encoding preferable to label encoding?
5. Why must scaling, imputation, and feature selection be fitted inside the training workflow?

Preparation for next lecture: Review linear equations, residuals, squared error, and the idea of minimizing a loss function. **Lecture 3** begins model construction with Linear Regression, Optimization, and Regularization.

Suggested Reading

Primary and supporting references

Primary reference

G. James, D. Witten, T. Hastie, and R. Tibshirani, *An Introduction to Statistical Learning with Applications in Python*, Springer, 2023.

- Chapter 2: Statistical Learning
- Review the role of inputs, outputs, model flexibility, training error, and test error.
- Use the data examples to connect exploratory analysis with later modeling.

Supporting references

A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, O'Reilly, 2022.

- Review the end-to-end ML project workflow, data cleaning, train/test splitting, and preprocessing pipelines.

K. P. Murphy, *Probabilistic Machine Learning: An Introduction*, MIT Press, 2022.

- Use selected introductory sections for a deeper view of data representations and probabilistic assumptions.

💡 When reading, focus on how preprocessing choices change the representation available to the model.